

Università
degli Studi di
Messina
DIPARTIMENTO DI INGEGNERIA

Master of **Engineering**
in Computer Science
A.Y. '25/'26

Industrial IoT

lab: RPL w/ Contiki

prof. Giovanni Merlino

RPL: The Routing Protocol for Low Power and Lossy Networks (LLNs)

- RPL is a specialized routing protocol designed for Low-Power and Lossy Networks (LLNs), commonly found in IoT applications.
- **Purpose:** Facilitates efficient data transmission across wireless sensor networks, smart grids, and other IoT ecosystems, optimizing for power consumption and network reliability.
- **Designed for LLNs:** Tailored to address the unique challenges of networks with constrained resources and high loss rates.
- **Scalability:** Supports large-scale network topologies, making it suitable for widespread IoT deployments.
- **Adaptability:** Dynamically adjusts to varying network conditions, enhancing reliability and performance.

Key Features of RPL

- **DODAG Structure:** Utilizes a Destination-Oriented Directed Acyclic Graph (DODAG) to organize network routes.
- **Multi-Path Routing:** Offers multiple routes for data transmission, reducing congestion and improving resilience.
- **Objective Function:** Employs configurable metrics to determine the best path for data packets, optimizing for energy efficiency, latency, or other criteria.

RPL Operation Modes and IPv6 Integration

- **Storing Mode:** Nodes store routing information to all destinations, suitable for networks with sufficient memory resources.

- **Non-Storing Mode:** Minimizes memory usage by storing route information primarily at the root node, leveraging source routing for data delivery.

RPL and IPv6 Integration:

- **IPv6 Support:** Fully compatible with IPv6, facilitating seamless integration into modern network infrastructures.

- **6LoWPAN Adaptation:** Works effectively with 6LoWPAN for header compression and efficient IPv6 packet transmission over low-power wireless networks.

Security in RPL

Built-in Security Features: Supports secure network operation with mechanisms for authentication, integrity protection, and confidentiality.
Adaptable to Security Needs: Flexible configuration options allow for tailored security policies, addressing specific network vulnerabilities.

Implementing RPL in IoT Projects

- **Built-in Security Features:** Supports secure network operation with mechanisms for authentication, integrity protection, and confidentiality.
- **Adaptable to Security Needs:** Flexible configuration options allow for tailored security policies, addressing specific network vulnerabilities.
- **Device Compatibility:** Compatible with a wide range of low-power IoT devices and microcontrollers.
- **Software Support:** Supported by numerous IoT operating systems and development frameworks, including Contiki-NG, RIOT, and others.

Introduction to Renode



- **What is Renode?** An open-source software development framework with commercial support from Antmicro, designed for developing, debugging, and testing multi-node device systems.
- **Purpose:** Enables reliable, scalable, and effective development across complex systems.
- **Support:** Offers both open-source accessibility and the option for commercial support, ensuring a wide range of applications from hobbyist projects to professional deployments.

Strengths of Renode



- **Determinism:** Full determinism of execution within a shared virtual time, crucial for consistent testing and debugging.
- **Debugging and Analysis:** Offers transparent and robust tools for debugging, tracing, and analysis, even in complex multi-node setups.
- **Integration:** Easily integrates with popular development tools and plugins, enhancing the development workflow.
- **Model Abstractions:** Features rich model abstractions that provide additional functionality, supported by a modular platform description format.
- **Automated Testing and CI:** Facilitates automated testing, continuous integration, and collaboration features.

Renode in Continuous Integration (CI)



- **CI in Edge Computing:** Addresses the gap in CI tools for complex edge systems, enhancing local development workflows with seamless CI capabilities.
- **CI Capabilities with Renode:** Allows tracking of repositories, running test pipelines, and gathering essential metrics and logs from test runs.
- **Local and Cloud Testing:** Supports testing on cloud or local servers, with easy commands to recreate setups for isolated and deterministic environments.

Key Features of Renode



- **Scalable Framework:** Designed with a focus on IoT, supporting rapid development and complex multinode scenarios.
- **Continuous Integration:** Built to enhance CI processes, improving testing capabilities and development efficiency.
- **Extensibility:** Open and easily extendable architecture, encouraging innovation and customization.
- **Security:** Enables the development of security-hardened devices and systems, addressing modern cybersecurity needs.

Getting Started with Renode

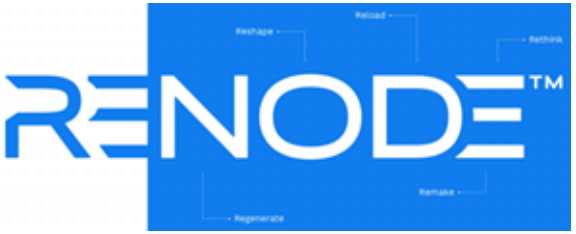


- **Available for Multiple Platforms:** Offers downloads for Debian, Fedora, Arch Linux, macOS, and Windows, ensuring broad accessibility.

- **Download Links:** Direct links to download Renode for various operating systems, facilitating easy setup and installation.

Renode represents a significant leap forward in the development and testing of multi-node device systems. Its focus on determinism, scalability, and integration with CI tools makes it a valuable asset for developers working on the edge of technology, particularly in the IoT space. Whether you're a solo developer or part of a larger team, Renode provides the tools and flexibility needed to bring complex systems to market faster and more reliably.

Renode Installation



- Why Renode? It provides a scalable workflow for creating tested, secure IoT systems by simulating unmodified software on virtual boards.
- Wide Architecture Support: ARMv7/v8, x86, RISC-V, SPARC, POWER, Xtensa.
- Comprehensive Simulation: Not just CPUs, but entire SoCs and network connections are simulated for real-world testing scenarios.

Installing Dependencies



- Mono/.NET Requirement: Mono >= 5.20 for Linux/macOS or .NET >= 4.7 for Windows.

```
sudo apt install ca-certificates gnupg
sudo gpg --homedir /tmp --no-default-keyring --keyring
/usr/share/keyrings/mono-official-archive-keyring.gpg --keyserver
hkp://keyserver.ubuntu.com:80 --recv-keys
3FA7E0328081BFF6A14DA29AA6A19B38D3D831EF
echo "deb [signed-by=/usr/share/keyrings/mono-official-archive-
keyring.gpg] https://download.mono-project.com/repo/ubuntu stable-
focal main" | sudo tee /etc/apt/sources.list.d/mono-official-
stable.list
sudo apt update
```

Installing Dependencies



- Linux Dependencies:

```
sudo apt-get install policykit-1 libgtk2.0-0 screen uml-utilities gtk-sharp2 libc6-dev gcc python3 python3-pip
```

- macOS and Windows: Follow specific instructions for each OS to ensure Renode runs smoothly.

Installing Renode from Packages



Debian
Linux, (.deb)
Download

Fedora
Linux, (.rpm)
Download

Arch
Linux, (.pkg.tar.xz)
Download

macOS
Download

Windows
Download

Exploring Contiki-NG for Next-Generation IoT Devices



- **Open-Source and Cross-Platform:** Contiki-NG is accessible and adaptable for a wide range of IoT devices.
- **Tailored for Next-Generation IoT Devices:** Designed with the future of IoT in mind, ensuring longevity and relevance.
- **Emphasizes Dependable Low-Power Communication:** Ideal for applications where power efficiency is crucial.
- **Incorporates Standard Protocols:** Includes IPv6/6LoWPAN, 6TiSCH, RPL, and CoAP, facilitating seamless integration and interoperability.
- **Robust Foundation for IoT Solutions:** Offers a secure and reliable platform for developing IoT applications.

```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG

Robust Security Measures:

- **Comprehensive Security:** Built-in support for secure communication with protocols like DTLS, ensuring data integrity and confidentiality across IoT devices.
- **Reliable Encryption:** Implements industry-standard encryption methods to protect data and communications in IoT networks.




```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG



Interoperability Through Standard Protocols:

- **IPv6/6LoWPAN Compatibility:** Facilitates internet connectivity and efficient data exchange in constrained environments.
- **6TiSCH Operation:** Leverages the Time Slotted Channel Hopping mechanism to ensure reliable, power-efficient communication in wireless industrial networks.
- **RPL Routing Support:** Implements the Routing Protocol for Low-Power and Lossy Networks (RPL) for efficient routing in mesh networks.
- **CoAP Integration:** Supports the Constrained Application Protocol (CoAP) for web-based interaction with IoT devices.

```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG

Efficient Power Management:

- **Low-Power Operation:** Optimized for energy efficiency, extending the battery life of IoT devices in the field.
- **Power Profiling Tools:** Includes tools for monitoring and analyzing power consumption, aiding in the development of power-efficient applications.




```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG

Cross-Platform Development Environment:

- **Simulator Support:** Provides a powerful simulator for testing and debugging IoT applications without the need for physical hardware.
- **Multi-architecture Compatibility:** Supports development for a wide range of microcontroller architectures, ensuring broad hardware compatibility.




```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG

Contiki-NG's technical architecture is designed to meet the demanding requirements of modern IoT ecosystems.

By emphasizing security, interoperability, and energy efficiency, Contiki-NG provides a solid foundation for developing a wide array of IoT applications, from smart home devices to industrial monitoring systems.

The platform's support for standard protocols and cross-platform development makes it a versatile choice for IoT developers looking to build reliable, future-proof solutions.




```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Technical Foundations of Contiki-NG

Contiki-NG's technical architecture is designed to meet the demanding requirements of modern IoT ecosystems.

By emphasizing security, interoperability, and energy efficiency, Contiki-NG provides a solid foundation for developing a wide array of IoT applications, from smart home devices to industrial monitoring systems.

The platform's support for standard protocols and cross-platform development makes it a versatile choice for IoT developers looking to build reliable, future-proof solutions.




```
git clone https://github.com/contiki-ng/contiki-ng.git
```

Running Contiki-NG in Renode





- Compatibility with Contiki-NG: Enables running, debugging, and testing Contiki-NG applications seamlessly, offering a real-hardware experience with added benefits of virtualization.
- Use Cases: Renode has been utilized by Antmicro, RISE SICS, and others for Contiki-NG development, demonstrating its versatility and efficacy.

https://github.com/contiki-ng/contiki-ng/blob/develop/doc/tutorials/Running-Contiki-NG-in-Renode.md

Initiating Renode from Contiki-NG



- Running Predefined Examples: Navigate to /examples/rpl-udp in the Contiki-NG tree and execute make renode TARGET=cc2538dk to start an RPL-UDP example with virtual server and client nodes.

- Custom Scripts: Use make renode TARGET=cc2538dk SCRIPT=<SCRIPT_NAME> to run custom scripts, adapting the simulation to your specific needs.



Initiating Renode from Contiki-NG





- Running Renode from Any Location:
- Starting Renode: Use the renode command to open the Monitor - Renode's command-line interface (CLI).
- Loading Demos: To load a Contiki-NG demo, use i @/path/to/contiki-ng/examples/rpl-udp/rpl-udp.resc, substituting the path to your specific demo.
- Modifying Binaries: Edit the $bin=@path_to_your_binary line in the script to change the binary being used in the simulation.

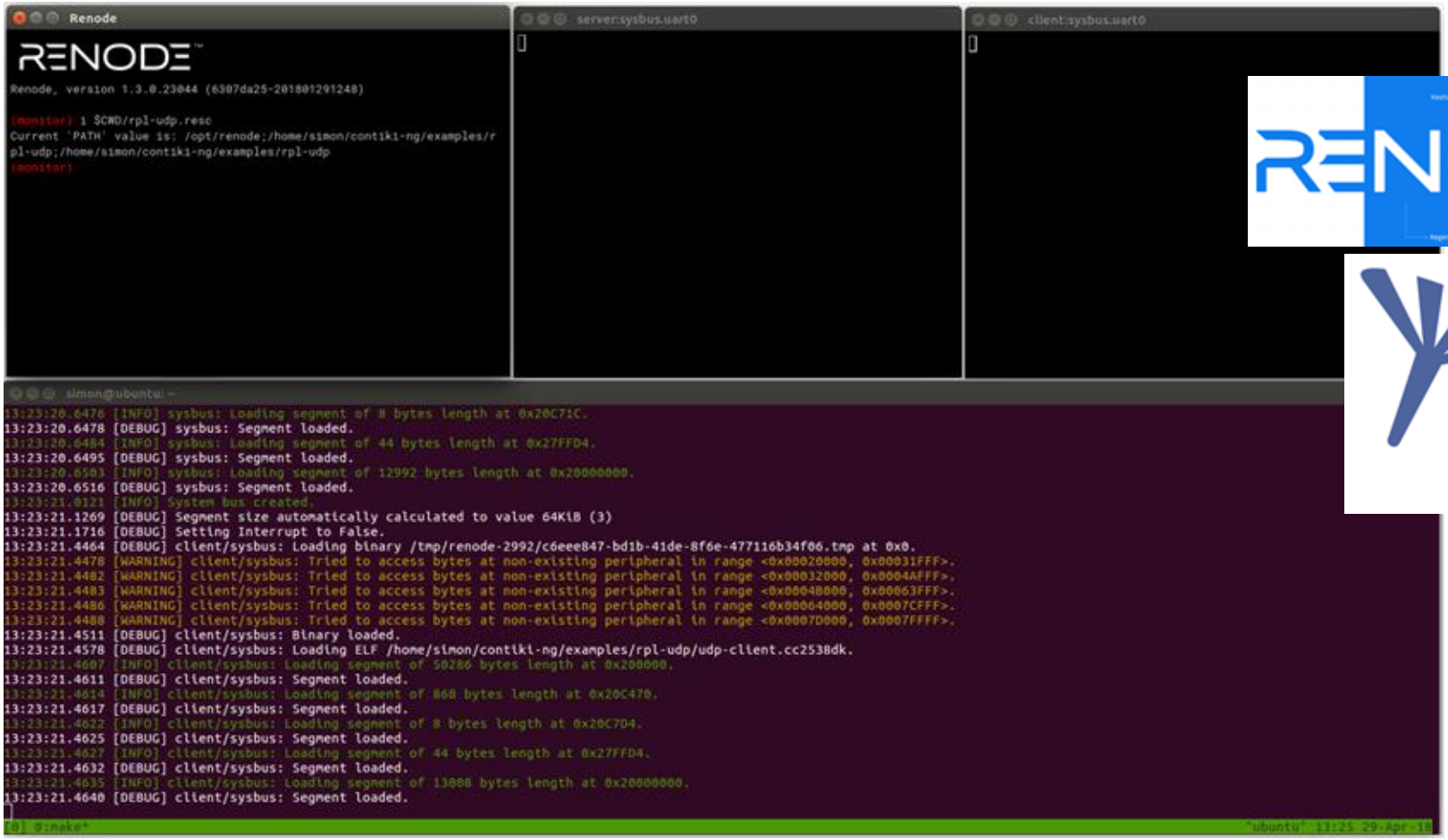
Customizing Simulations in Renode

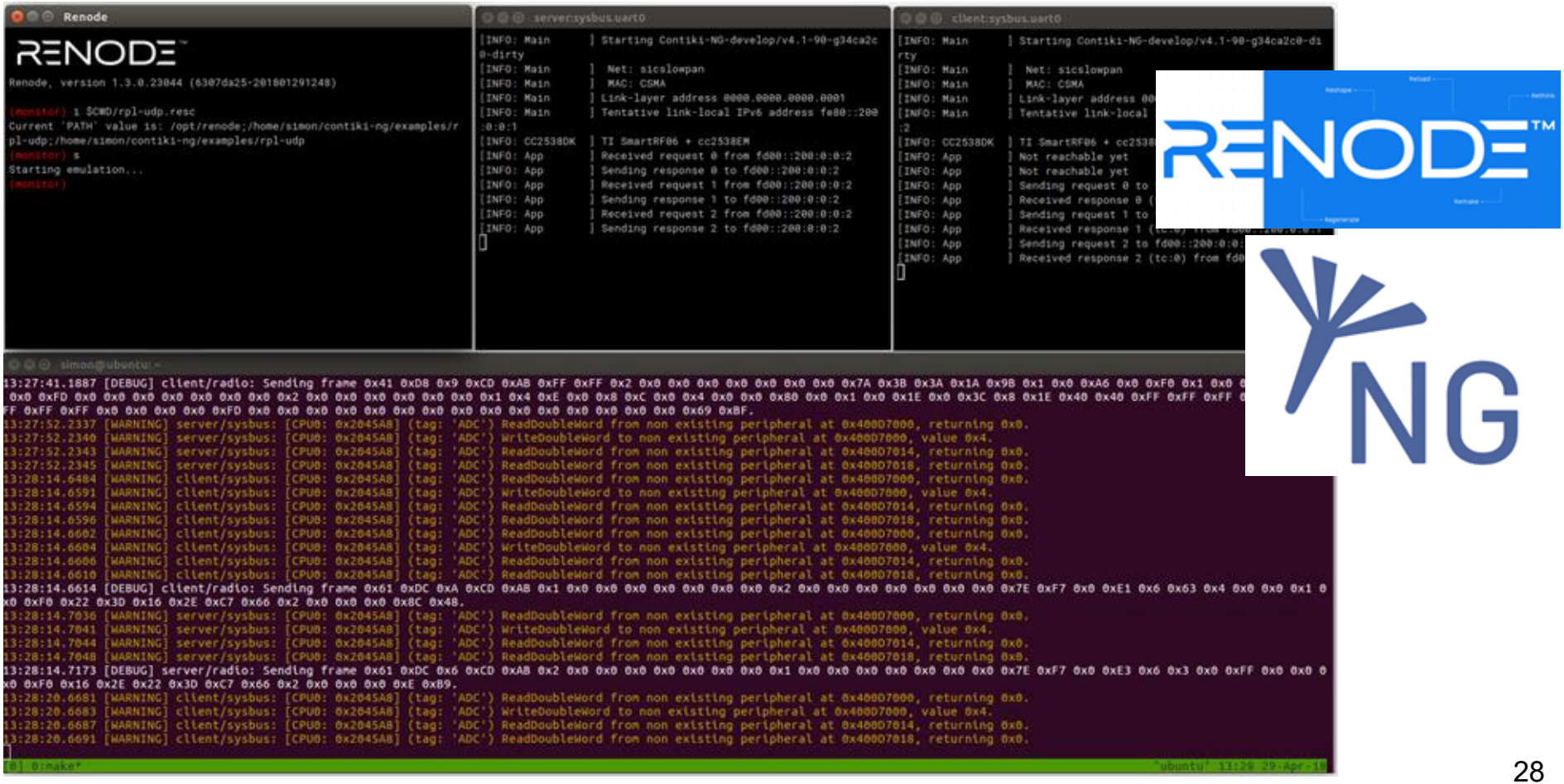




- Script Modification: Adjust the script path with i $ORIGIN/cc2538-node.resc to reference the correct machine script located in the current directory ($ORIGIN).
- Dynamic Binary Changes: The $bin variable can be updated multiple times within a single script to affect subsequent machines, allowing for flexible configurations.

Renode provides a powerful platform for developing and testing IoT applications built with Contiki-NG. By simulating real-world scenarios in a controlled, virtual environment, developers can enhance the reliability, security, and performance of their IoT solutions. Whether running predefined examples or customizing simulations, Renode accelerates the development cycle and facilitates collaboration among team members.


Renode
RENODE
Renode, version 1.3.0.23044 (6307da25-201801291248)
(monitor) i $CWD/rpl-udp.resc
Current 'PATH' value is: /opt/renode;/home/simon/contiki-ng/examples/rpl-udp;/home/simon/contiki-ng/examples/rpl-udp
(monitor)
server:sysbus.uart0
client:sysbus.uart0
simon@ubuntu: ~
13:23:20.6476 [INFO] sysbus: Loading segment of 8 bytes length at 0x20C71C.
13:23:20.6478 [DEBUG] sysbus: Segment loaded.
13:23:20.6484 [INFO] sysbus: Loading segment of 44 bytes length at 0x27FFD4.
13:23:20.6495 [DEBUG] sysbus: Segment loaded.
13:23:20.6503 [INFO] sysbus: Loading segment of 12992 bytes length at 0x20000000.
13:23:20.6516 [DEBUG] sysbus: Segment loaded.
13:23:21.0121 [INFO] System bus created.
13:23:21.1269 [DEBUG] Segment size automatically calculated to value 64KiB (3)
13:23:21.1716 [DEBUG] Setting Interrupt to False.
13:23:21.4464 [DEBUG] client/sysbus: Loading binary /tmp/renode-2992/c6eee847-bd1b-41de-8f6e-477116b34f06.tmp at 0x0.
13:23:21.4478 [WARNING] client/sysbus: Tried to access bytes at non-existing peripheral in range <0x00020000, 0x00031FFF>.
13:23:21.4482 [WARNING] client/sysbus: Tried to access bytes at non-existing peripheral in range <0x00032000, 0x0004AFFF>.
13:23:21.4483 [WARNING] client/sysbus: Tried to access bytes at non-existing peripheral in range <0x00048000, 0x00063FFF>.
13:23:21.4486 [WARNING] client/sysbus: Tried to access bytes at non-existing peripheral in range <0x00064000, 0x0007CFFF>.
13:23:21.4488 [WARNING] client/sysbus: Tried to access bytes at non-existing peripheral in range <0x0007D000, 0x0007FFFF>.
13:23:21.4511 [DEBUG] client/sysbus: Binary loaded.
13:23:21.4578 [DEBUG] client/sysbus: Loading ELF /home/simon/contiki-ng/examples/rpl-udp/udp-client.cc2538dk.
13:23:21.4607 [INFO] client/sysbus: Loading segment of 50286 bytes length at 0x200000.
13:23:21.4611 [DEBUG] client/sysbus: Segment loaded.
13:23:21.4614 [INFO] client/sysbus: Loading segment of 868 bytes length at 0x20C470.
13:23:21.4617 [DEBUG] client/sysbus: Segment loaded.
13:23:21.4622 [INFO] client/sysbus: Loading segment of 8 bytes length at 0x20C7D4.
13:23:21.4625 [DEBUG] client/sysbus: Segment loaded.
13:23:21.4627 [INFO] client/sysbus: Loading segment of 44 bytes length at 0x27FFD4.
13:23:21.4632 [DEBUG] client/sysbus: Segment loaded.
13:23:21.4635 [INFO] client/sysbus: Loading segment of 13008 bytes length at 0x20000000.
13:23:21.4640 [DEBUG] client/sysbus: Segment loaded.
[0] 0:make*
"ubuntu" 13:25 29-Apr-18
RENODE™
NG


Renode
RENODE
Renode, version 1.3.0.23044 (6307da25-201801291248)
(monitor) i $CWD/rpl-udp.resc
Current 'PATH' value is: /opt/renode:/home/simon/contiki-ng/examples/rpl-udp:/home/simon/contiki-ng/examples/rpl-udp
(monitor) s
Starting emulation...
(monitor)
server:sysbus.uart0
[INFO: Main] Starting Contiki-NG-develop/v4.1-90-g34ca2c0-dirty
[INFO: Main] Net: sicslowpan
[INFO: Main] MAC: CSMA
[INFO: Main] Link-layer address 0000.0000.0000.0001
[INFO: Main] Tentative link-local IPv6 address fe80::200:0:0:1
[INFO: CC2538DK] TI SmartRF06 + cc2538EM
[INFO: App] Received request 0 from fd00::200:0:0:2
[INFO: App] Sending response 0 to fd00::200:0:0:2
[INFO: App] Received request 1 from fd00::200:0:0:2
[INFO: App] Sending response 1 to fd00::200:0:0:2
[INFO: App] Received request 2 from fd00::200:0:0:2
[INFO: App] Sending response 2 to fd00::200:0:0:2
client:sysbus.uart0
[INFO: Main] Starting Contiki-NG-develop/v4.1-90-g34ca2c0-dirty
[INFO: Main] Net: sicslowpan
[INFO: Main] MAC: CSMA
[INFO: Main] Link-layer address 00
[INFO: Main] Tentative link-local
:2
[INFO: CC2538DK] TI SmartRF06 + cc2538
[INFO: App] Not reachable yet
[INFO: App] Not reachable yet
[INFO: App] Sending request 0 to
[INFO: App] Received response 0
[INFO: App] Sending request 1 to
[INFO: App] Received response 1
[INFO: App] Sending request 2 to fd00::200:0:0:
[INFO: App] Received response 2 (tc:0) from fd0
simon@ubuntu: ~
13:27:41.1887 [DEBUG] client/radio: Sending frame 0x41 0xD8 0x9 0xCD 0xAB 0xFF 0xFF 0x2 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x7A 0x3B 0x3A 0x1A 0x9B 0x1 0x0 0xA6 0x0 0xF0 0x1 0x0 0x0 0xFD 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x2 0x0 0x0 0x0 0x0 0x0 0x0 0x1 0x4 0xE 0x0 0x8 0xC 0x0 0x4 0x0 0x0 0x80 0x0 0x1 0x0 0x1E 0x0 0x3C 0x8 0x1E 0x40 0x40 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0x0 0x0 0x0 0x0 0xFD 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x69 0xBF.
13:27:52.2337 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7000, returning 0x0.
13:27:52.2340 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') WriteDoubleWord to non existing peripheral at 0x400D7000, value 0x4.
13:27:52.2343 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7014, returning 0x0.
13:27:52.2345 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7018, returning 0x0.
13:28:14.6484 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7000, returning 0x0.
13:28:14.6591 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') WriteDoubleWord to non existing peripheral at 0x400D7000, value 0x4.
13:28:14.6594 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7014, returning 0x0.
13:28:14.6596 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7018, returning 0x0.
13:28:14.6602 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7000, returning 0x0.
13:28:14.6604 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') WriteDoubleWord to non existing peripheral at 0x400D7000, value 0x4.
13:28:14.6606 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7014, returning 0x0.
13:28:14.6610 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7018, returning 0x0.
13:28:14.6614 [DEBUG] client/radio: Sending frame 0x61 0xDC 0xA 0xCD 0xAB 0x1 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x2 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x7E 0xF7 0x0 0xE1 0x6 0x63 0x4 0x0 0x0 0x1 0x0 0xF0 0x22 0x3D 0x16 0x2E 0xC7 0x66 0x2 0x0 0x0 0x0 0x8C 0x48.
13:28:14.7036 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7000, returning 0x0.
13:28:14.7041 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') WriteDoubleWord to non existing peripheral at 0x400D7000, value 0x4.
13:28:14.7044 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7014, returning 0x0.
13:28:14.7048 [WARNING] server/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7018, returning 0x0.
13:28:14.7173 [DEBUG] server/radio: Sending frame 0x61 0xDC 0x6 0xCD 0xAB 0x2 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x1 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x7E 0xF7 0x0 0xE3 0x6 0x3 0x0 0xFF 0x0 0x0 0x0 0xF0 0x16 0x2E 0x22 0x3D 0xC7 0x66 0x2 0x0 0x0 0x0 0xE 0xB9.
13:28:20.6681 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7000, returning 0x0.
13:28:20.6683 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') WriteDoubleWord to non existing peripheral at 0x400D7000, value 0x4.
13:28:20.6687 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7014, returning 0x0.
13:28:20.6691 [WARNING] client/sysbus: [CPU0: 0x2045A8] (tag: 'ADC') ReadDoubleWord from non existing peripheral at 0x400D7018, returning 0x0.
[0] 0:make*
"ubuntu" 13:28 29-Apr-18
RENODE™
NG

Script:






```
:name: cc2538 range medium 3 nodes

################################################################## Sample script creating 1 server node and 2 client node s using a
# range-based medium function - transmission is possible if nodes are
# in range. Nodes are positioned in a 3D space (x,y,z),
# at a distance of 10 (units) from each other.
##################################################################emulation CreateIEEE802_15_4Medium "wireless"

# The radio is using a range-based medium (with the `Range` set to `11`)
# If not set, the default SimpleMediumFunction will be used (where range is not relevant)
wireless SetRangeWirelessFunction 11

######################## UDP SERVER - begin ########################
$bin=@https://dl.antmicro.com/projects/renode/udp-server.elf-s_173732-a2aefb896d521a8fdc2a052cac1933c68cc39bf5
$name="server"
i @scripts/single-node/cc2538.resc
connector Connect radio wireless
wireless SetPosition radio 0 0 0
mach clear
########################## UDP SERVER - end ##########################

######################## UDP CLIENT - begin ########################
$bin=@https://dl.antmicro.com/projects/renode/udp-client.elf-s_173840-9eed7fe31993d055b98410b886044e8205a95644
$name="client-1"
i @scripts/single-node/cc2538.resc
connector Connect radio wireless
wireless SetPosition radio 10 0 0
mach clear
########################## UDP CLIENT - end ##########################

######################## UDP CLIENT - begin ########################
$name="client-2"
i @scripts/single-node/cc2538.resc
connector Connect radio wireless
wireless SetPosition radio 0 10 0
mach clear
########################## UDP CLIENT - end ##########################
```

Understanding Renode Scripts: cc2538 Range Medium Example

`https://github.com/renode/renode/blob/master/scripts/multi-node/cc2538/rpl-udp.resc`

This Renode script simulates a network with 1 server node and 2 client nodes within a specific range, demonstrating a common IoT scenario.

Setting Up the Simulation Environment

Creating the Medium: emulation CreateIEEE802_15_4Medium "wireless" initializes a wireless medium for IEEE 802.15.4 radio communication.

Range-based Medium Function: wireless SetRangeWirelessFunction 11 defines the range within which nodes can communicate, set to 11 units.

Understanding Renode Scripts: cc2538 Range Medium Example

Configuring the UDP Server Node

Binary and Name: Loads the server binary from a provided URL and names the machine "server".

Script Inclusion: i @scripts/single-node/cc2538.resc includes the cc2538 single-node script for configuring the server node.

Positioning: Sets the server's position in the 3D space to 0 0 0, serving as the reference point for the simulation.

Understanding Renode Scripts: cc2538 Range Medium Example

Adding the First UDP Client Node

Client Binary: Loads the client binary from a URL, preparing the node to run as "client-1".

Node Configuration: Reuses the single-node script for cc2538 to set up the client node.

Spatial Positioning: Positions the first client node at 10 0 0, making it 10 units away from the server in the x-axis.

Understanding Renode Scripts: cc2538 Range Medium Example

Integrating the Second UDP Client Node

Naming: Assigns "client-2" as the name for the second client node, distinguishing it from the first client.

Repeating Setup Steps: Similar steps as the first client, including script inclusion and radio connection.

Different Position: Places the second client at 0 10 0, 10 units away from the server but in the y-axis this time.

Understanding Renode Scripts: cc2538 Range Medium Example

Key Takeaways from the Script

Modularity: Demonstrates how Renode scripts can reuse components like node setup scripts to streamline simulation setup.

Range Functionality: Showcases the use of range-based communication, essential for testing network protocols in spatially distributed systems.

3D Positioning: Highlights the capability to simulate nodes in 3D space, offering a realistic approximation of physical environments.